



Mansoura University
Faculty of Computers and Information Sciences
Department of Computer Science
First Semester- 2024-2025



[CS212P/IT212] Computer Organization and Architecture

Grade : 2nd General

Dr. Ghada Shafiq

CONVERT NUMBER FROM IEEE FLOATING-POINT FORMAT TO DECIMAL

- Ex: 1 10000011 110000000000000000000000

Solution :

- Sign = 1 → **negative**
- Convert biased exponent to decimal then subtract **127** from it to get the actual exponent. (10000011) → (131).
 $131 - 127 = 4$
- Take the mantissa (1100) with the hidden bit (1 before the dot) then write it as **1.1100×2^4**
- Finally, convert the number (0.1100) to decimal, then multiply it by 2^4
- Final result: **$1.75 \times 2^4 = - 28$**

BINARY ARITHMETIC

Addition: $(110101010)_2 + (101101100)_2 = (?)_2$

$$\begin{array}{r} \\ \\ + \\ \hline \end{array}$$

BINARY ARITHMETIC

Subtraction: $(11001011)_2 - (1101110)_2 = (?)_2$

$$\begin{array}{r} \\ \\ \\ - \\ \hline \end{array}$$

Diagram illustrating binary subtraction: $(11001011)_2 - (1101110)_2$. The result is $(01011101)_2$.

BINARY SUBTRACTION WITH 2'S COMPLEMENT

- 2's complement method will be used to subtract two binary numbers.
- The operation is carried out by means of the following steps:
 - At first, 2's complement of the subtrahend is found.
 - Then it is added to the minuend.
 - If the final carry over of the sum is 1, it is dropped, and the result is positive.
 - If there is no carry over, the two's complement of the sum will be the result, and it is negative.

BINARY SUBTRACTION WITH 2'S COMPLEMENT

- **Evaluate: 110110 - 010110**

- Result:

1 1 0 1 1 0

1 0 1 0 1 0 (2's complement)

=====

Carry over 1 1 0 0 0 0 0 → final answer : 1 0 0 0 0 0 (drop the carry)

BINARY SUBTRACTION WITH 2'S COMPLEMENT

- **Evaluate: 10110 – 11010**

- Result:

1 0 1 1 0

0 0 1 1 0 (2's complement)

=====

no Carry over 1 1 1 0 0 → final answer : 0 0 1 0 0 (2's complement of the result)

BINARY ARITHMETIC

Multiplication: $(1011)_2 \times (1100)_2 = (?)_2$

					1	0	1	1
				×	1	1	0	0
					0	0	0	0
			0		0	0	0	
		1		0	1	1		
	1	0		1	1			
					1	0	0	0
1	0	0	0	0	0	1	0	0

Division: $(110101)_2 \div (111)_2 = (?)_2$

				0	1	1	1
1	1	1		1	1	0	1
			-	0	0	0	
				1	1	0	1
			-	1	1	1	
				1	1	0	0
			-	1	1	1	
				1	0	1	1
			-	1	1	1	
				1	0	0	

BINARY ARITHMETIC

SIGNED MAGNITUDE - 1'S AND 2'S COMPLEMENT

- Signed magnitude: Generally, the MSB is the sign bit, and the convention is that when the sign bit is 0, the number represented is positive and when the sign bit is 1, the number is negative.
- 1's complement is the inversion of bits while 2's complement is generated via adding 1 to the 1's complement.
- Signed magnitude, 1's and 2's complement using 7-bits:

	Signed Magnitude	1's Complement	2's Complement
23	0010111	0010111	0010111
-23	1010111	1101000	1101001

ENDIANNESS

- When we talk about computer memory, we refer to it as a collection of bytes, each of which is assigned a unique address.
- In computing, the order in which these bytes are stored is called **byte ordering**, and there are two different ways to do it: big-endian (BE) and little-endian (LE).

ENDIANNESS

Big Endian

Address

0x100

0x101

0x102

0x103

0x12

0x34

0x56

0x78

Data

0x12345678

Little Endian

Address

0x100

0x101

0x102

0x103

0x78

0x56

0x34

0x12

ENDIANNESS

- In **big-endian** byte order, the most significant byte (MSB) is stored at the smallest address, while the least significant byte (LSB) is stored at the largest address. This means that the byte ordering is from left to right, with the most significant byte first.
- In contrast, **little endian** byte order stores the LSB at the smallest address and the MSB at the largest address. This means that the byte ordering is from right to left, with the least significant byte first.

EXAMPLE

- Fill in the following table to show how the given integers are represented, assuming 16-bits are used to store values and the machine uses 2's complement notation

Integer	Binary	Hex	4 Byte Big Endian (hex value as seen in memory)	4 Byte Little Endian (hex value as seen in memory)
28				
2216				
-18675				
-12				

EXAMPLE

- Fill in the following table to show how the given integers are represented, assuming 16-bits are used to store values, and the machine uses 2's complement notation

Integer	Binary	Hex	4 Byte Big Endian (hex value as seen in memory)	4 Byte Little Endian (hex value as seen in memory)
28	Answer: 0000000000011100	Answer: 001C	Answer: 001C	Answer: 1C00
2216	Answer: 0000100010101000	Answer: 08A8	Answer: 08A8	Answer: A808
-18675	Answer: 1011011100001101	Answer: B70D	Answer: B70D	Answer: 0DB7
-12	Answer: 111111111110100	Answer: FFF4	Answer: FFF4	Answer: F4FF

EXAMPLE

Address	Value
100	2A
101	C2
102	08
103	1B

- Consider a 32-bit hexadecimal number stored in memory as follows:
 - (a) If the machine is big endian and uses 2's complement representation for integers, write the 32-bit integer number stored at address 100 (you may write the number in hex).
 - (b) If the machine is big endian and the number is an IEEE single-precision floating point value, is the number positive or negative?
 - (c) If the machine is big endian and the number is an IEEE single-precision floating point value, determine the decimal equivalent of the number stored at address 100 (you may leave your answer in scientific notation form, as a number times a power of two)
 - (d) If the machine is little endian and uses 2's complement representation for integers, write the 32-bit integer number stored at address 100 (you may write the number in hex).

CONT.

Address	Value
100	2A
101	C2
102	08
103	1B

- Consider a 32-bit hexadecimal number stored in memory as follows:
 - (e) If the machine is little endian and the number is an IEEE single-precision floating point value, is the number positive or negative?
 - (f) If the machine is little endian and the number is an IEEE single-precision floating point value, determine the decimal equivalent of the number stored at address 100 (you may leave your answer in scientific notation form, as a number times a power of two).

ANSWERS

(a) 0x2AC2081B

(b) 2A implies first bit is zero so it is positive

(c) Answer: IEEE single precision has 1 sign bit, 8 exp bits (bias of 127), and 23 bits for the significand. So, we have:

2AC2081B = 0010 1010 1100 0010 0000 1000 0001 1011

which can be written: $1.100001000001000000110112 \times 2^{-42}$

or approximately: $1.51587210 \times 2^{-42}$

ANSWERS

(d) 0x1B08C22A

(e) 1B implies first bit is zero so it is positive

(f) Answer: IEEE single precision has 1 sign bit, 8 exp bits (bias of 127), and 23 bits for the significand. So, we have:

1B08C22A = 0001 1011 0000 1000 1100 0010 0010 1010

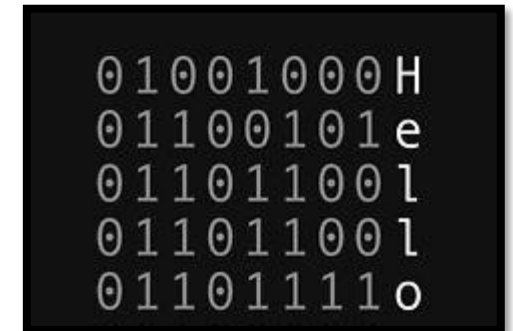
which can be written: $1.000100011000010001010102 \times 2^{-73}$

or approximately: $1.06842510 \times 2^{-73}$

CHARACTER ENCODING - ASCII

- Characters are "encoded" (or "represented") using a chosen "character encoding schemes"
- **ASCII** (American Standard Code for Information Interchange)
- Uses a 7-bit encoding scheme, which means it represents characters with values from 0 to 127.
- Includes basic Latin characters: English alphabet (uppercase and lowercase), digits, and common punctuation marks.
- Limited to English characters and control codes, so it lacks support for non-Latin alphabets.
- ASCII is also a subset of many larger encoding schemes, like UTF-8, which maintains backward compatibility with ASCII.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
10	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
20	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL



CHARACTER ENCODING - UNICODE

- **UNICODE** (Universal Coded character set)
- Unicode aims to provide a standard character encoding scheme, which is universal, efficient, and unambiguous.
- It has over 1,114,112 code points to every letter in every language.

<https://www.babelstone.co.uk/Unicode/unicode.html>



UNICODE TRANSFORMATION FORMAT (UTF)

- **UTF** refers to encoding formats that map Unicode code points to actual binary data, allowing them to be stored in memory, transmitted over networks, or displayed on screens.

	H	e	l	l	o
Unicode	U+0048	U+0065	U+006C	U+006C	U+006F
UTF-8	48	65	6C	6C	6F
UTF-16	00 48	00 65	00 6C	00 6C	00 6F
ASCII	48	65	6C	6C	6F

TYPES OF UTF

- **UTF-8:** Uses variable-length encoding (1 to 4 bytes).

It's efficient for English and other Latin-based languages (1 byte per character) but can expand to 4 bytes for characters in less common scripts.

- **UTF-16:** Uses 2 or 4 bytes, making it more efficient for characters requiring multiple bytes in UTF-8 but less efficient for English.
- **UTF-32:** Uses a fixed 4 bytes for each character, making it simple but memory-intensive.

Example: The character "A" (code point U+0041) is stored as **0x41** in **UTF-8** (1 byte),
0x0041 in **UTF-16** (2 bytes),
and **0x00000041** in **UTF-32** (4 bytes).

HOW TO UTF-8 ?

Unicode Range	Binary Code Point Length	UTF-8 Encoding Pattern	Bytes Used	Example	UTF-8 Bytes
U+0000 to U+007F	$bits \leq 7$	0xxxxxxx	1 byte	U+0041 ('A')	41
U+0080 to U+07FF	$7 < bits \leq 11$	110xxxxx 10xxxxxx	2 byte	U+00A9 ('©')	C2 A9
U+0800 to U+FFFF	$11 < bits \leq 16$	1110xxxx 10xxxxxx 10xxxxxx	3 byte	U+20AC ('€')	E2 82 AC
U+10000 to U+10FFFF	$16 < bits \leq 21$	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	4 byte	U+1F600 ('😄')	F0 9F 98 80

THANKS ♥